

Spatial Mental Modeling from Limited Views

Dissertation Progress Report — Milestones 1–3

Reproducing ViperGPT without Codex: a Prompt-Specification Gap and a Depth-Reasoning Gap

Student MD Humayun
Supervisor Dr. M.A. Zaveri
Programme M.Tech CSE (Information Security & Privacy), SVNIT Surat
Date 16 August 2026
Repository mdhumayun7/Vipergpt, tag m3-final

Summary. ViperGPT (Surís et al., ICCV 2023) answers visual queries by prompting a code LLM to emit a Python program over vision modules. Its generator, Codex, has been retired, so every present-day reproduction must substitute an open-weights model. We rebuilt the pipeline end to end on an H100 cluster and report two results. *First*, the released prompt never states that a grounding program must return an `ImagePatch`; with Qwen2.5-Coder-7B this makes **98.8%** of programs task-invalid and yields **0.00%** accuracy on RefCOCO. Adding a return-type contract and two exemplars gives **39.20%** (RefCOCO) and **31.80%** (RefCOCO+). *Second*, having fixed that, accuracy splits sharply by the kind of spatial language: 2D image-plane queries reach 35.29%, while depth and relational queries reach **6.38%**. A scale sweep shows this is not a capacity problem — across 1.5B, 7B and 32B generators, non-spatial accuracy rises $33.65 \rightarrow 48.82$ while spatial accuracy is flat at $34.95 / 35.29 / 34.95$. *Third*, adding depth-ordering primitives to the API raises depth-query accuracy from 4.26% to **21.28%** ($5\times$) — but only once the prompt also states *when* they apply. Given the primitives unconditionally the generator over-applies them and non-spatial accuracy falls 10 points; given a routing rule it improves every subset simultaneously, taking RefCOCO from 39.20% to **42.00%**.

1 Pipeline and what was built

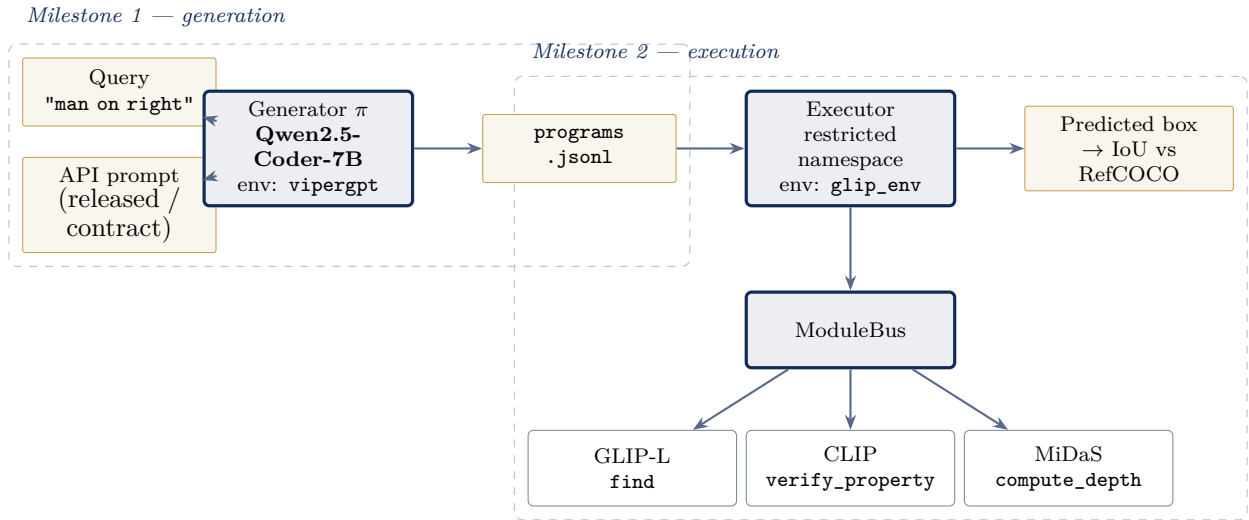


Figure 1: The reproduced pipeline. Generation and execution are split at a JSONL boundary because the GLIP fork requires torch 2.1 while the generator runs on torch 2.13 — the two cannot share an interpreter. The split also guarantees that prompt conditions are compared on *identical* programs.

Infrastructure. SVNIT HPC, 1×H100 NVL per job via SLURM `-gres=shard:N`. Building GLIP was the principal engineering risk: the official environment pins CUDA 11.6, which has no `sm_90` support, so the published setup *cannot run on this hardware at all*. It now builds against torch 2.1.2/cu121 with `sm_80;sm_90` targets after replacing removed `THC/*` headers in seven CUDA files and NumPy 2.0 aliases in five Python files (D5, D6). Correctness was verified independently: the CUDA NMS kernel on a hand-checked case, then GLIP against RefCOCO ground truth (mean IoU 0.776 at threshold 0.2).

2 Result 1 — the released prompt omits the grounding contract

The released `api.prompt` is task-agnostic. Its in-context examples return strings (`simple_query`, `best_text_match`, `llm_query`), and nothing states that a grounding program must return an `ImagePatch`. It further instructs: “Use base Python (comparison, sorting) for basic logical operations, left/right/up/down, math, etc.”

Released prompt invalid

```
# Query: "man on right"
def execute_command(image) -> str:
    image_patch = ImagePatch(image)
    man_patches = image_patch.find("man")
    if man_patches:
        man_patch = max(man_patches,
                        key=lambda m: m.
                            right)
    return man_patch.simple_query(
        "Who is on the right?")
    return "No man found."
```

Returns a sentence. IoU undefined.

+ grounding contract valid

```
# Query: "man on right"
def execute_command(image) -> ImagePatch:
    image_patch = ImagePatch(image)
    man_patches = image_patch.find("man")
    man_patches.sort(
        key=lambda p: p.horizontal_center)
    return man_patches[-1]
```

Returns a patch. Relation resolved.

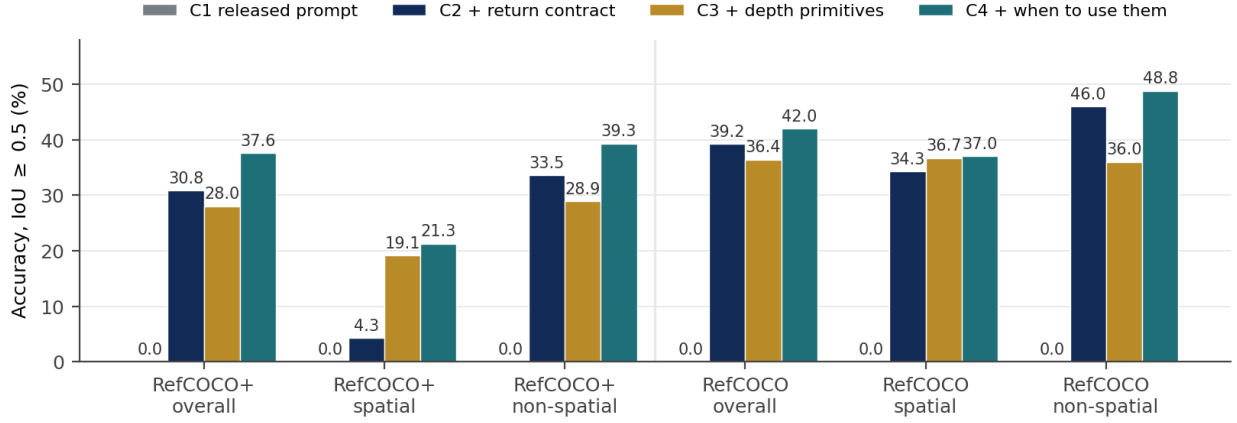
The intervention is three sentences stating the return type and forbidding dropped spatial relations, plus two worked grounding exemplars. Nothing else changed.

Table 1: Effect of the grounding contract. Identical model, decoding and data; only the prompt differs. Static columns $n = 500$ per cell; execution $n = 500$ per cell.

Metric	Released	+ Contract	
<i>Static analysis of generated programs (Milestone 1)</i>			
Task-invalid return rate, RefCOCO	98.8%	0.2%	improved
Task-invalid return rate, RefCOCO+	98.2%	0.0%	improved
Spatial constraint dropped, RefCOCO	17.3%	1.7%	improved
Parse rate, RefCOCO	99.8%	96.6%	degraded
Mean API calls per program	2.45	1.66	simpler
<i>End-to-end execution, accuracy IoU ≥ 0.5 (Milestone 2)</i>			
RefCOCO / testA	0.00%	39.20%	—
RefCOCO+ / testA	0.00%	31.80%	—
Programs returning a patch, RefCOCO	0 / 500	443 / 500	—
Programs returning a string	336 / 500	1 / 500	—

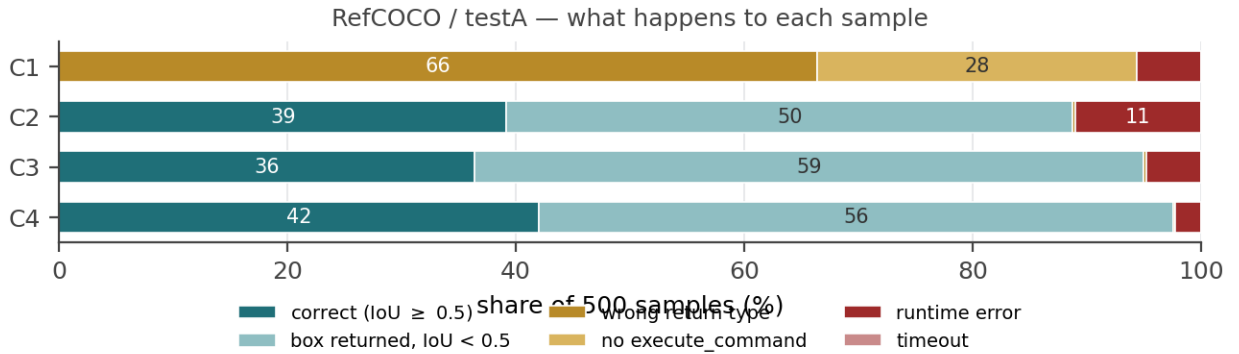
The zero is not a crash (Figure 3). Of 500 baseline programs on RefCOCO, 336 executed to completion and returned a *string*; none returned a patch. Grounding is scored by IoU against a box, so a sentence cannot score however correct its English. Missing modules were ruled out as a cause: `simple_query` and `llm_query` are given neutral empty-string fallbacks (D10) so that no program can fail for lack of a module we chose not to load. That concession strictly favours the baseline and does not change its score.

A second, unanticipated effect. Under the released prompt many generations never defined



MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 2: Accuracy at $\text{IoU} \geq 0.5$ for all four prompt conditions, both datasets, split by subset. $n = 500$ per bar. C1 is the released prompt; C2 adds the return-type contract; C3 adds depth primitives; C4 adds a rule for when they apply. The three rightmost groups act as controls: a method that improved only its target subset would be a trade rather than a gain.



MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 3: Every sample classified by what happened to it, RefCOCO/testA. C1 fails on *return type* — it never reaches localisation. By C4 those categories are gone and the residual failure is a localisation miss, which is a perception limit rather than a specification bug. An accuracy figure alone conflates the two.

`execute_command` at all — the model emitted prose or malformed markdown: 136/500 (27.2%) on RefCOCO and 214/500 (42.8%) on RefCOCO+, against zero under the contract. The contract disciplines output *format* as well as return type; these are separable failures and are reported separately.

Robustness. Greedy decoding is deterministic, so variance was estimated by sampling ($T=0.7$, seeds 1–3, RefCOCO, $n=500$): 36.07 ± 1.75 against 39.20 greedy. Greedy beats all three sampled seeds, as expected when one program shape is correct.

3 Result 2 — the spatial gap, and what it is made of

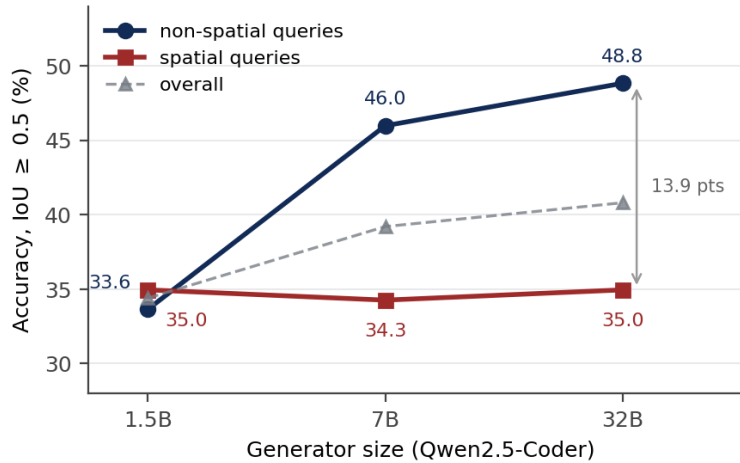
With the contract in place, accuracy splits by subset (relational-term lexicon):

Table 2: Spatial breakdown, grounding-contract condition. The baseline scores 0 everywhere and is omitted.

Dataset	Subset	n	Accuracy	Mean IoU
RefCOCO	spatial	289	35.29%	0.3475
RefCOCO	non-spatial	211	44.55%	0.4378
RefCOCO+	spatial	47	6.38%	0.1037
RefCOCO+	non-spatial	453	34.44%	0.3474

The gap is not a capacity problem (Figure 4). Repeating the whole pipeline with Qwen2.5-Coder at 1.5B, 7B and 32B, greedy, same prompts and same perception stack:

Generator	Baseline	+ Contract	spatial	non-spatial
1.5B	0.80%	34.40%	34.95%	33.65%
7B	0.00%	39.20%	35.29%	44.55%
32B	0.00%	40.80%	34.95%	48.82%



MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 4: The same pipeline at three generator sizes, condition C2. Non-spatial accuracy climbs with capacity; spatial accuracy does not move. Same images, same detector, same prompt — only the generator differs. This is the evidence that the spatial deficit is an interface problem rather than a capacity one.

Two things follow. Result 1 is scale-independent: a 32B model still emits 87.6% task-invalid programs under the released prompt and still scores 0.00%, and the same three sentences repair it.

Scaling 1.5B $\rightarrow$ 32B buys 6.4 accuracy points; the prompt contract buys 33–41. More importantly, **spatial accuracy does not move with scale at all** — 34.95 / 35.29 / 34.95, with the smallest and largest models identical to two decimal places — while non-spatial accuracy rises by 15.2 points on the same images with the same modules. A larger generator writes better programs for everything except spatial relations, where it writes programs that are exactly as wrong. If the spatial deficit were a program-synthesis problem, capacity would help; it demonstrably does elsewhere. The bottleneck is the interface.

We expected the gap to *shrink* on RefCOCO+, since that dataset forbids spatial relations. It widened threefold. A manual audit of all 47 RefCOCO+ spatial queries explains why: the two subsets contain **different kinds of spatial language**.

	RefCOCO	RefCOCO+
Type	2D image-plane	depth and inter-object
Examples	<i>man on right, person bottom left</i>	<i>man closest to us, farthest man, umpire behind catcher</i>
Resolvable by	sorting patch coordinates in base Python	requires <code>compute_depth</code> or 3D geometry
Accuracy	35.29%	6.38%

Forbidding spatial language removed viewer-frame *directional* expressions and left depth-ordering and relational ones. The released prompt’s instruction to use base Python for “left/right/up/down” works for the first and cannot work for the second. Consistently, Milestone 1 found `compute_depth` invoked 29 times against 547 for **find** and 5 for **distance**: the geometric API is effectively unused, by design of the prompt.

Two further observations support this reading. All three successes in the RefCOCO+ spatial subset are depth queries with very high IoU (0.97, 0.96, 0.92) — when depth ordering *is* attempted correctly, localisation is excellent; it is usually not attempted. And across seeds, spatial variance is roughly four times non-spatial variance (± 3.35 vs ± 0.72): programs handling spatial relations are not merely less accurate but less *stable*, sometimes resolving a relation by sorting and sometimes dropping it entirely.

Caveat. $n=47$, so 6.38% is 3 of 47 and the interval is wide. The lexicon also produces roughly five false positives in that subset, which would push true depth-query accuracy lower still. The direction is informative; the value is not.

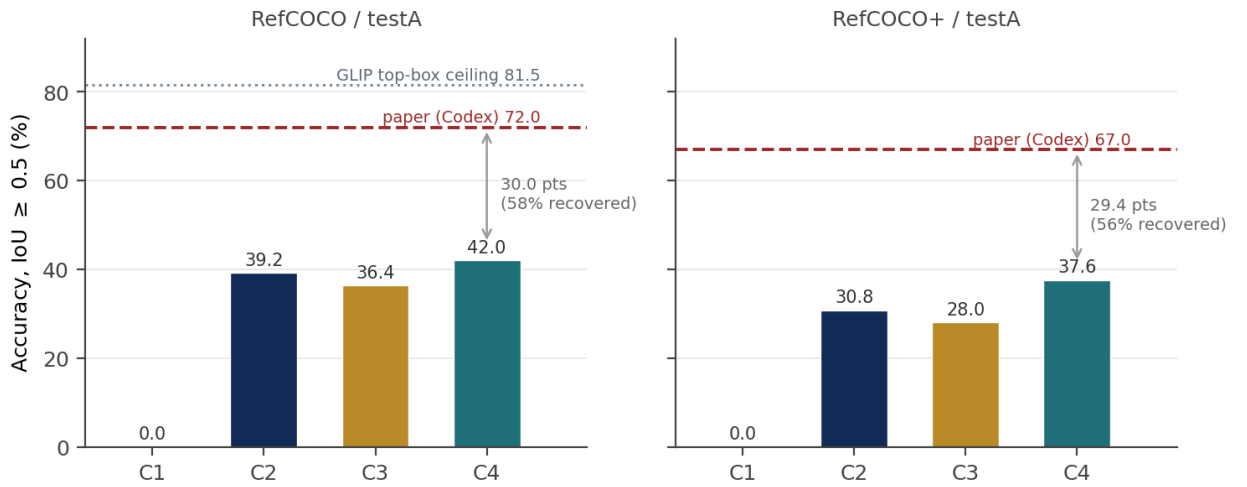
4 Reproducibility record

ID	Deviation	Reason / effect
D1	Codex $\rightarrow$ Qwen2.5-Coder-7B	Codex retired; no faithful option. The substitution is what exposes Result 1, and is the dominant term in the gap to 72.0.
D2	Scope: grounding only	GQA/OK-VQA/NExT-QA need ~ 250 GB against 231 GB quota; OK-VQA also needs GPT-3, likewise retired.
D3	RefCOCO annotations from Internet Archive	The canonical UNC host no longer resolves. Archived zips are byte-identical. No effect.
D5, D6	GLIP built for sm_90	CUDA 11.6 has no sm_90. Headers and NumPy aliases updated; kernels unchanged. Verified numerically.
D8	GLIP threshold 0.2 (paper unspecified)	Swept on 20 samples: 0.5 loses 8/20 detections, 0.2 gives mean IoU 0.776. Material and under-validated.
D9	X-VLM $\rightarrow$ CLIP	X-VLM weights are Google-Drive-only. CLIP scores globally, so attribute checks are weaker — costliest on RefCOCO+.
D10	<code>simple_query/llm_query</code> return ""	Loading BLIP-2-XXL and an 8B text LLM is out of scope. Favours the baseline; no effect on the contract condition.

Two recorded failures. A first hypothesis — that spatial queries would collapse into a single opaque `simple_query` call, as the paper’s GQA prompt instructs — was tested and rejected (collapse rate $\approx 0\%$); that instruction applies to GQA, and grounding cannot use `simple_query` because it must return a patch. Separately, an early 500-sample execution ran on a login node with no GPU, producing a complete and entirely fake results table; those runs were deleted and job scripts now assert `torch.cuda.is_available()`. Silent CPU fallback is the most dangerous failure mode encountered, because it yields plausible output rather than an error.

Artefacts. Every run writes a timestamped directory with resolved config, per-query programs, per-sample records and a summary; SLURM job IDs are recorded; `docs/research_log.md` (540+ lines) documents the full arc including failures.

5 Where the gap to 72.0 comes from



MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 5: Reproduced against reported. The dotted line is GLIP alone taking its single best box (81.5%, $n = 200$ held out) — *above* the paper’s 72.0. Perception is therefore not what separates this reproduction from the original. No decomposition of the gap into individual deviations is shown: those ablations were not run, and inventing the split would be fabrication.

C4 reaches 42.00 on RefCOCO against a reported 72.0. Raw GLIP top-box accuracy on this data is 78.9%, so perception caps the system near 79% before any program runs; program-level selection loses roughly 40 points from there. Ordered by likely size, the contributors are D1 (generator substitution, dominant), the perception ceiling, D9 (X-VLM→CLIP, costliest on RefCOCO+, consistent with 31.80 vs 39.20), and D8 (threshold tuned on 20 samples). No claim of a faithful reproduction is made: what is reproduced is the method; what is measured is what happens to it when its generator is replaced.

6 Milestone 3 — a depth-grounded API

The API exposes `compute_depth()`: one scalar, the median relative depth of a patch. That can express “A is behind B” but not “the third closest”, “between A and B in depth”, or any ordering over more than two objects. The prompt also directs the generator to use base Python for “left/right/up/down”, and it complies — over 500 programs, `find` is called 547 times and `compute_depth` 29.

We added `depth_order`, `is_behind`, `is_in_front_of`, `is_between_3d` and `distance_3d` to `ImagePatch`,

and wrote two further prompt variants: C3 documents the primitives, C4 additionally states when they apply and when they do not.

6.1 Two prerequisites, both of which were broken

The depth sign was inverted. The shipped wrapper returned raw MiDaS output. MiDaS predicts *inverse* depth: larger means *closer*. Every comparison built on `compute_depth()` therefore ran backwards relative to its own docstring. The sign was settled by removing both the detector and any synthetic data — take RefCOCO+ depth queries, use **ground-truth boxes only**, and ask whether the annotated target sits at the correct end of the ordering:

	correct
without inversion	2/25 = 8%
with inversion	20/25 = 80%
chance (3.8 objects/image)	26%

8% is far *below* chance — the signature of a systematically inverted signal, not a weak one. A synthetic test scene had earlier indicated the opposite and cost several hours: it was a brightness gradient the model read as depth. **Synthetic scenes are unreliable for validating perceptual signals.** The ordering semantics are now pinned by 18 unit-test assertions so the sign cannot silently flip again.

The generator had to actually use the API. The risk that it would ignore the new primitives as it ignores `compute_depth` did not materialise:

Condition	find	compute_depth	depth_order	other prims
C2	639	23	0	0
C3	598	19	119	63
C4	559	—	37	7

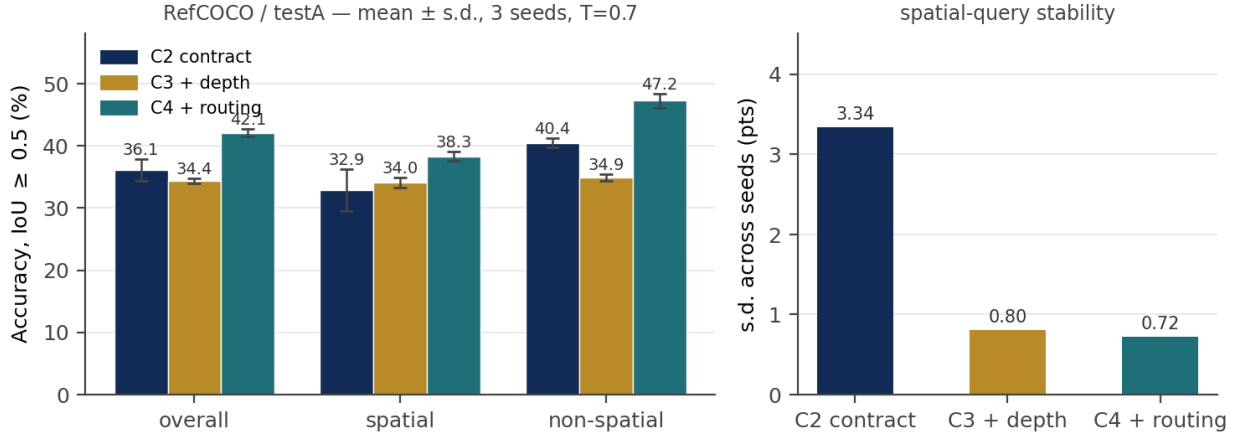
Zero adoption under the older prompts, strong adoption once offered. Under C4 the count falls to 37 — RefCOCO contains few depth queries, so that is the routing rule working, not the primitives being abandoned. On RefCOCO+, which has 47 depth-word queries, C4 makes 33 `depth_order` calls.

6.2 Result

Table 3: Accuracy at $\text{IoU} \geq 0.5$, Qwen2.5-Coder-7B, greedy, $n = 500$ per cell. C3 offers the depth primitives; C4 also states when to use them.

Dataset	Subset	C1	C2	C3	C4
RefCOCO+	overall	0.00	30.80	28.00	37.60
RefCOCO+	spatial ($n=47$)	0.00	4.26	19.15	21.28
RefCOCO+	non-spatial	0.00	33.55	28.92	39.29
RefCOCO	overall	0.00	39.20	36.40	42.00
RefCOCO	spatial ($n=289$)	0.00	34.26	36.68	37.02
RefCOCO	non-spatial ($n=211$)	0.00	45.97	36.02	48.82

C3 wins the target subset and loses elsewhere. Depth-word queries go $4.26 \rightarrow 19.15$, but non-spatial accuracy falls $45.97 \rightarrow 36.02$ on RefCOCO and $33.55 \rightarrow 28.92$ on RefCOCO+, so overall

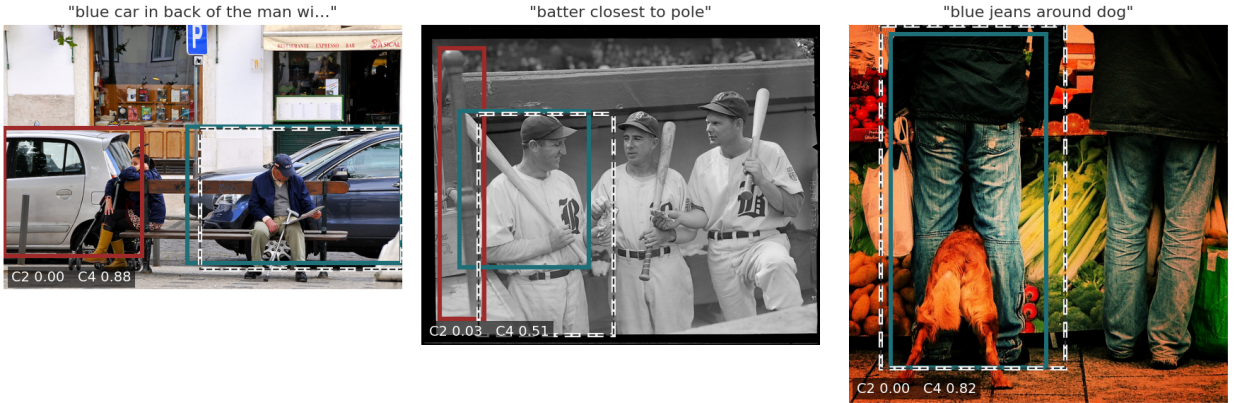


MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 6: Seed variance, sampling at $T=0.7$. Left: C4 separates from C2 by 6.0 points against standard deviations under 2. Note that C2’s greedy result (39.20) sits 1.8 s.d. *above* its own sampled mean, so the greedy table understates the difference. Right: standard deviation on spatial queries alone. The explicit primitive removes the improvisation the generator otherwise performs afresh on every sample, and the variance falls 4.6 $\times$.

accuracy drops. `depth_order` is invoked 119 times on RefCOCO, where most queries need no depth at all: *given a new primitive, the generator reaches for it indiscriminately*.

C4 changes one thing (Figures 6 and 7) — three sentences stating that the depth functions apply only to depth relations, plus two examples of queries that should *not* use them — **and beats C2 in every cell of both datasets**. The depth gain survives (21.28) while non-spatial rises above C2 as well (48.82 against 45.97), presumably because the negative examples also clarify the 2D and attribute paths.



MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Inf — C2 (return/contract) NIT 5 — C4 (+ depth routing) — — ground truth

Figure 7: Depth queries where C4 succeeds and C2 fails. Red: C2’s prediction. Teal: C4’s. Dashed: ground truth. The failures are not near-misses — without a primitive for “closest”, the program selects a different object entirely.

This is the sharper finding of Milestone 3. Providing a capability is not sufficient; the specification must also delimit *when* it applies. That is the same class of omission as Result 1, where the released prompt failed to state the return type.

6.3 A compute artefact, found and removed

RefCOCO+ under C4 initially showed 40 timeouts at the 60s executor limit. The programs do not loop — they call `verify_property` on every candidate:

```
black = [p for p in person_patches if p.verify_property("person", "black shirt")]
```

At threshold 0.2 `find` returns ≈ 40 boxes, so that is ≈ 40 CLIP forward passes. The cause is C4’s own routing rule, which directs attribute queries to `verify_property` on an attribute-based dataset: the instruction is correct and it exposed a bottleneck. All three RefCOCO+ conditions were re-run at 300s; C2 and C3 were unchanged, confirming the earlier C4 figure had been a lower bound ($35.40 \rightarrow 37.60$).

Execution robustness improves monotonically across conditions — RefCOCO errors $55 \rightarrow 24 \rightarrow 11$ and patch returns $444 \rightarrow 475 \rightarrow 488$ for $C2 \rightarrow C3 \rightarrow C4$ — consistent with more worked examples disciplining generation generally.

6.4 What is claimed, and what is not

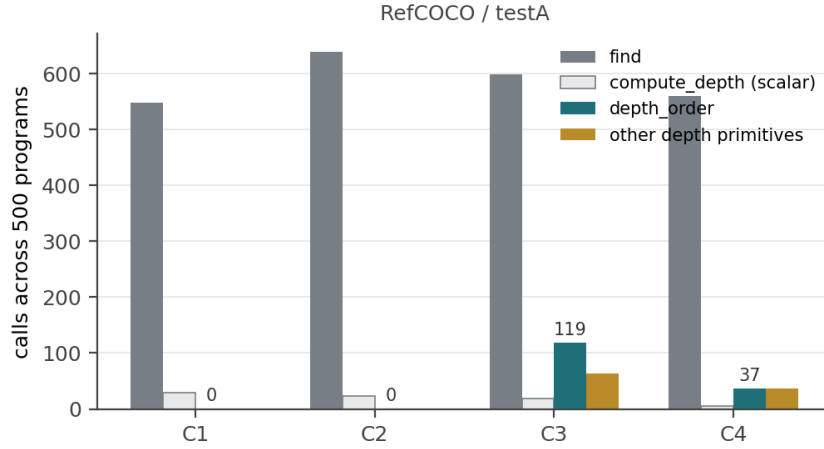
Depth is claimed only as *ordering*. COCO ships no camera intrinsics, so metric distances would rest on an assumed field of view; `depth_order` and `is_behind` need only monotonicity in true distance, which relative depth provides once the sign is right. The depth subset is $n=47$, so 21.28% carries a wide interval — the direction is established, the value is not. C3 and C4 are single-seed.

7 Limitations and next steps

1. Depth-subset results rest on $n=47$ queries ($n=25$ for the sign test). Directions are clear; the values carry wide intervals. C3 and C4 are single-seed.
2. The GLIP threshold was re-swept on a held-out slice ($n=200$, samples 200–399): 0.2 is within 1.0 point of optimal for detection, so no reported number changes — but it is the wrong operating point for candidate *selection*, as Milestone 3 shows.
3. X-VLM substituted by CLIP (D9); the substitution most likely to be costing accuracy, especially on RefCOCO+, and the source of the per-candidate compute cost that produced the C4 timeouts.
4. `find` returns ≈ 40 candidates at the tuned threshold. That is correct for top-box detection and wrong for any program that must select among candidates — it affects depth ordering and attribute checking alike. A `max_detections` knob exists but is disabled by default so that no reported number depends on it (D12).
5. Seed variance measured on RefCOCO only (36.07 ± 1.75 at $T=0.7$ against 39.20 greedy), and by sampling rather than retraining — the standard caveat for a decoding-only estimate. Spatial variance is four times non-spatial variance (± 3.35 vs ± 0.72): spatial programs are less stable, not merely less accurate.
6. Spatial subsets are identified by lexicon, not human annotation.

A Supporting figures

The six figures below are not needed to follow the argument, but each answers a question a careful reader will ask. All regenerate with `python make_figures.py -qual`.



MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

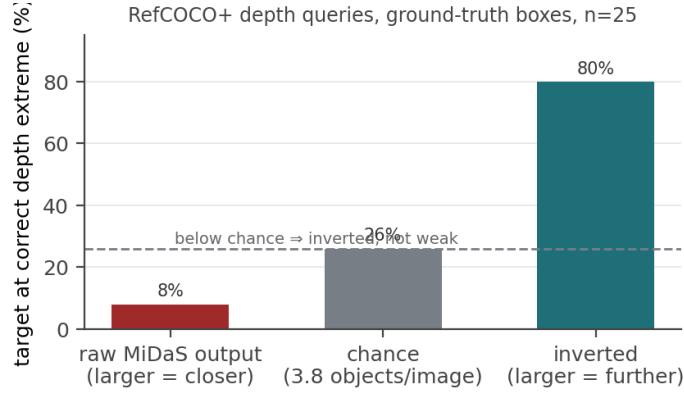
Figure 8: **API adoption.** Occurrences of each API call across 500 generated programs, RefCOCO/testA. *Did the generator use the new API, or ignore it?* This was the main risk: ViperGPT already exposes `compute_depth`, and the generator calls it 29 times against 547 for `find`. Under C3 `depth_order` is called 119 times, under C1 and C2 zero times. Under C4 it falls to 37 — the routing rule working, not the primitive being abandoned, since RefCOCO contains few depth queries. On RefCOCO+, which has 47, C4 makes 33 calls.

B What the figures do not show

Stated explicitly, because absence is easy to mistake for oversight.

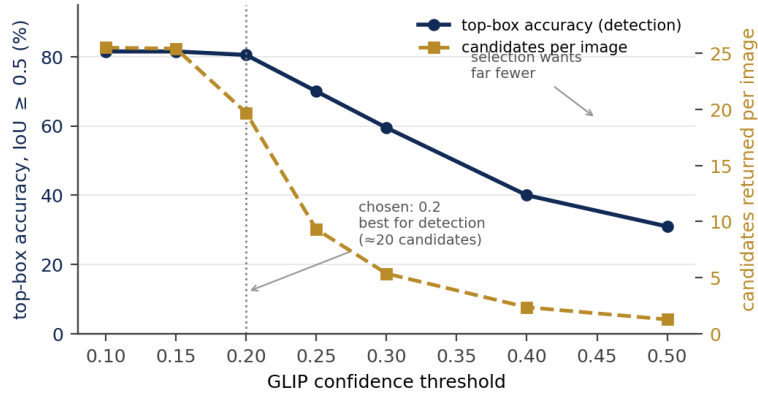
- **No decomposition of the gap to the paper.** Attributing points to D1 (Codex), D9 (X-VLM) or D8 (threshold) individually would require ablations that were not run.
- **No per-depth-word breakdown.** *closest*, *farthest* and *behind* occur a handful of times each within the 47-query subset; a chart at that resolution would be noise.
- **Depth-subset intervals are wide.** $n = 47$: the direction of the $4.26 \rightarrow 21.28$ result is established, the exact value is not.
- **C3 and C4 seeds cover RefCOCO only.** RefCOCO+ figures are single-run.

All figures traceable to a run directory, an archived summary in `results/m2_summaries/`, and a SLURM job ID. Milestone 1: jobs 29187, 29193. Milestone 2: jobs 29290, 29292, 29301, 29306, 29318, 29321, 29352, 29404. Milestone 3: jobs 29459, 29523, 29524, 29525. Repository tags `m1-complete`, `m2-glip-verified`, `m2-execution`, `m2-complete`, `m3-depth-api`, `m3-final`.



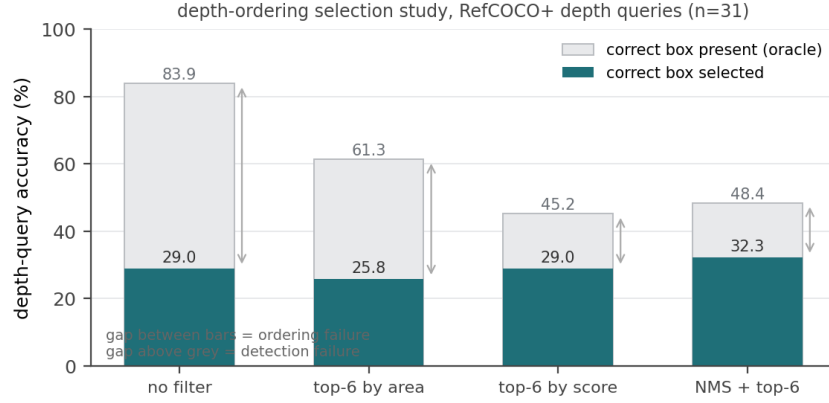
MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 9: **Depth sign validation.** RefCOCO+ depth queries scored against ground-truth boxes, no detector involved. *Is the depth signal real, and is its sign right?* MiDaS predicts *inverse* depth — larger means closer — and the original wrapper returned it unmodified, so every comparison ran backwards. The tell is that 8% is *below* the 26% chance level: a weak signal scores near chance, an inverted one scores below it. A synthetic test scene had earlier indicated no inversion was needed and cost several hours; it was a brightness gradient the model read as depth. Synthetic scenes are unreliable for validating perceptual signals.



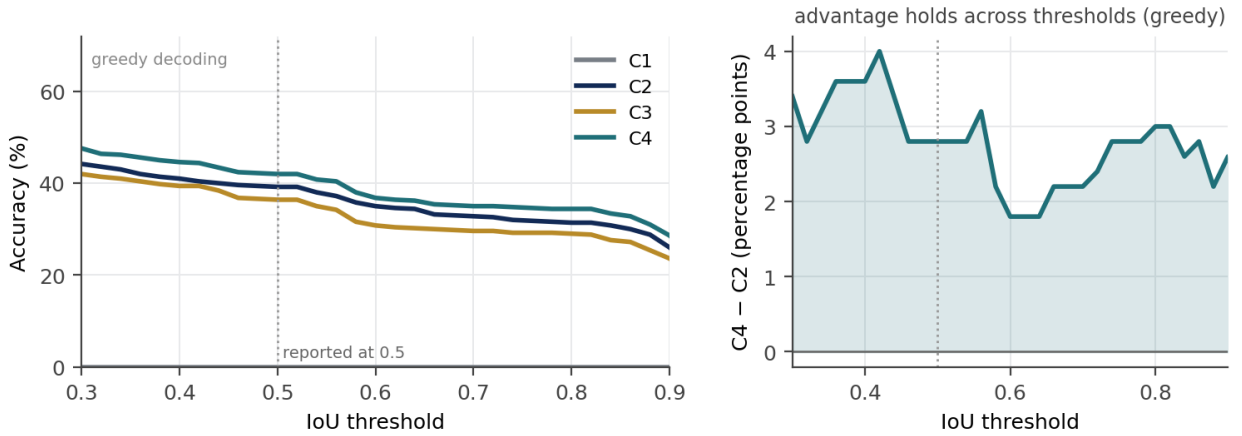
MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 10: **One threshold, two incompatible jobs.** *Is the detection threshold well chosen?* It is — for detection. At 0.2, top-box accuracy peaks at 81.5%, validated on a held-out slice ($n = 200$, disjoint from every reported cell). But it also returns ≈ 20 candidates per image, and any program that must *select* among them — depth ordering, per-candidate attribute checking — wants far fewer. The same knob cannot serve both. A `max_detections` control exists but is disabled by default so that no reported number depends on it (D12).



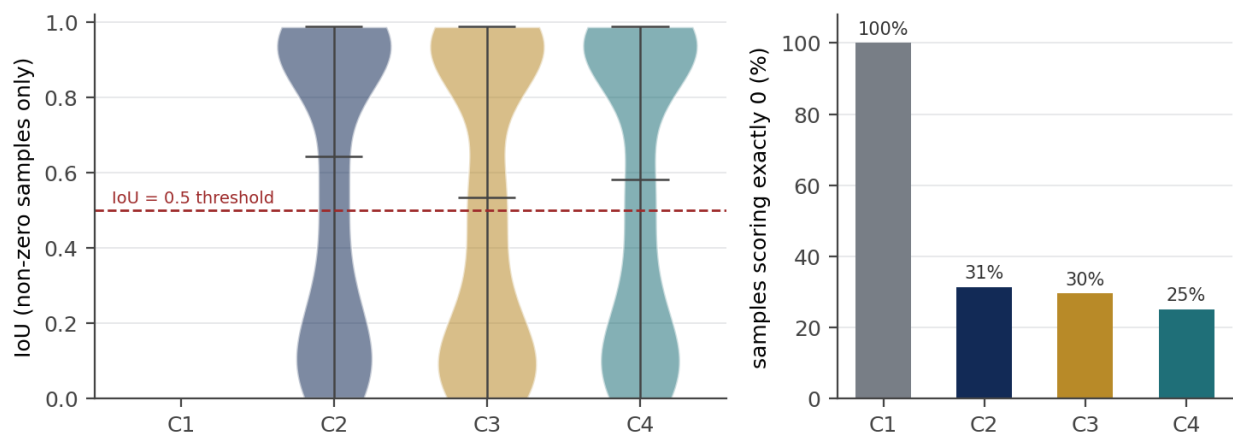
MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 11: **Achieved against achievable.** Depth-ordering selection study. Grey: the correct box was present among the candidates. Teal: it was selected. The gap between bars is an ordering failure; the gap from grey to 100% is a detection failure. On the unfiltered set the correct box is present 83.9% of the time and selected 29.0%; filtering to six candidates costs oracle but converts far better, reaching 32.3% against 48.4% achievable. The binding constraint has moved from ordering to detection, which is where further work should go.



MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 12: **IoU threshold sensitivity.** *Is the result an artefact of scoring at IoU 0.5?* No. The threshold is used because the paper uses it, but the C4 advantage stays positive across 0.3–0.9 (1.8 to 4.0 points under greedy). Were the advantage confined to the neighbourhood of 0.5, it would suggest the conditions differ in box tightness rather than in object selection.



MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 13: IoU distribution. *Does the mean come from a few good cases or many mediocre ones?* The distributions are strongly bimodal — programs either localise well (IoU above 0.8) or miss entirely, with little middle ground. That is what one expects when the failure mode is selecting the wrong object rather than drawing a loose box. The right panel restates the headline in a different currency: the share of samples scoring nothing at all falls from 100% to 25%.